CHAPTER 4

Factory Method Pattern

This chapter covers the Factory Method pattern.

GoF Definition

Define an interface for creating an object, but let subclasses decide which class to instantiate. The Factory Method pattern lets a class defer instantiation to subclasses.

Note To understand this pattern, I suggest you refer to Chapter 24, which covers the Simple Factory pattern. The Simple Factory pattern does not fall directly into the Gang of Four design patterns, so the discussion of that pattern appears in Part II of the book. The Factory Method pattern will make more sense to you if you can understand the pros and cons of the Simple Factory pattern first.

Concept

The concept can be best described with the following examples.

Real-Life Example

In a restaurant, based on customer inputs, a chef varies the taste of dishes to make the final products.

© Vaskaran Sarcar 2018
V. Sarcar, *Design Patterns in C#*, https://doi.org/10.1007/978-1-4842-3640-6_4

www.EBooksWorld.ir

Computer World Example

In an application, you may have different database users. For example, one user may use Oracle, and the other may use SQL Server. Whenever you need to insert data into your database, you need to create either a `SqlConnection` or an `OracleConnection` and only then can you proceed. If you put the code into `if-else` (or `switch`) statements, you need to repeat a lot of code, which isn't easily maintainable. This is because whenever you need to support a new type of connection, you need to reopen your code and make those modifications. This type of problem can be resolved using the Factory Method pattern. Here I'll provide an abstract creator class (`IAnimalFactory`) to define the basic structure. As per the definition, the instantiation process will be carried out through the subclasses that derive from this abstract class.

Illustration

I have created all the classes in a single file. There is no need to create separate folders.

Class Diagram

Figure 4-1 shows the class diagram.

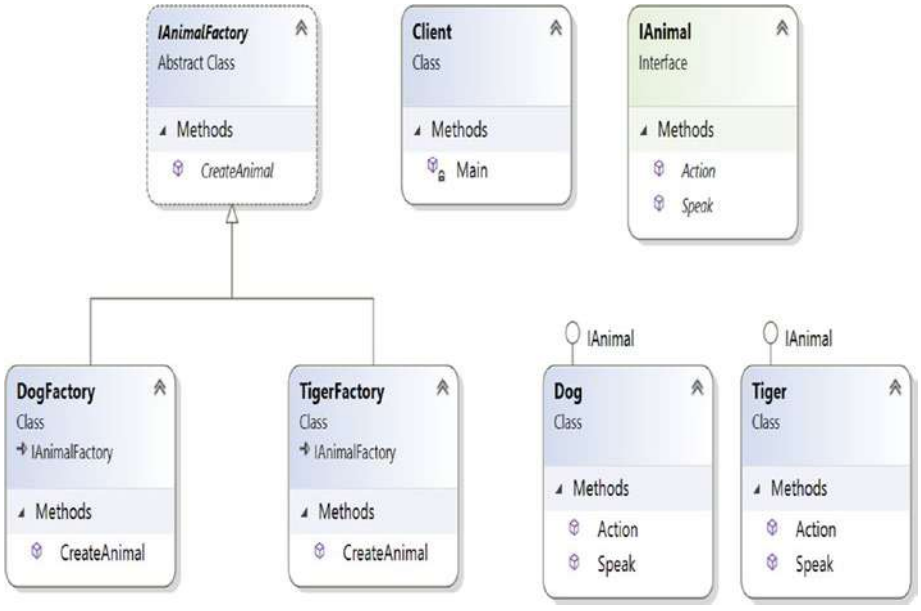


Figure 4-1. *Class diagram*

www.EBooksWorld.ir

Directed Graph Document

Figure 4-2 shows the directed graph document.

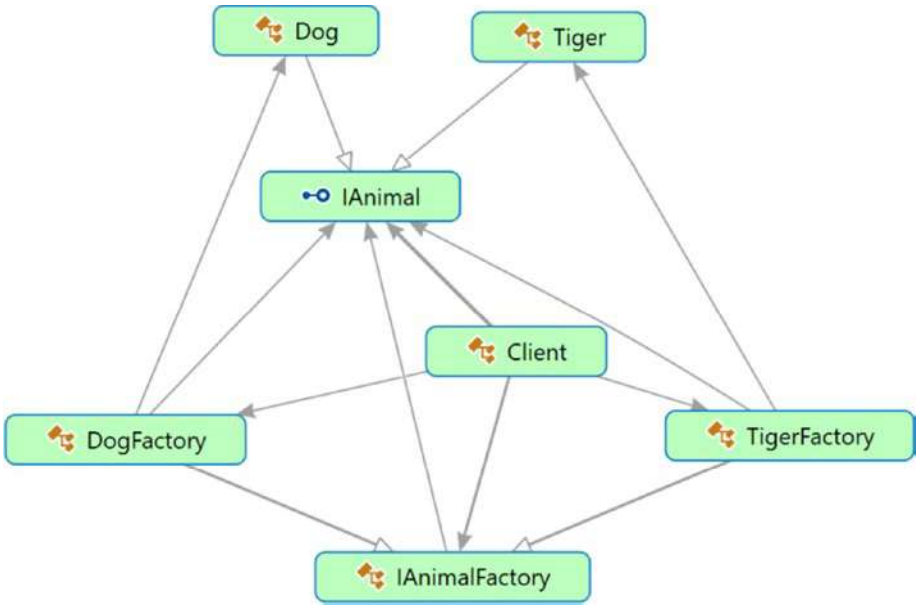


Figure 4-2. *Directed Graph Document*

Solution Explorer View

Figure 4-3 shows the high-level structure of the parts of the program.

www.EBooksWorld.ir

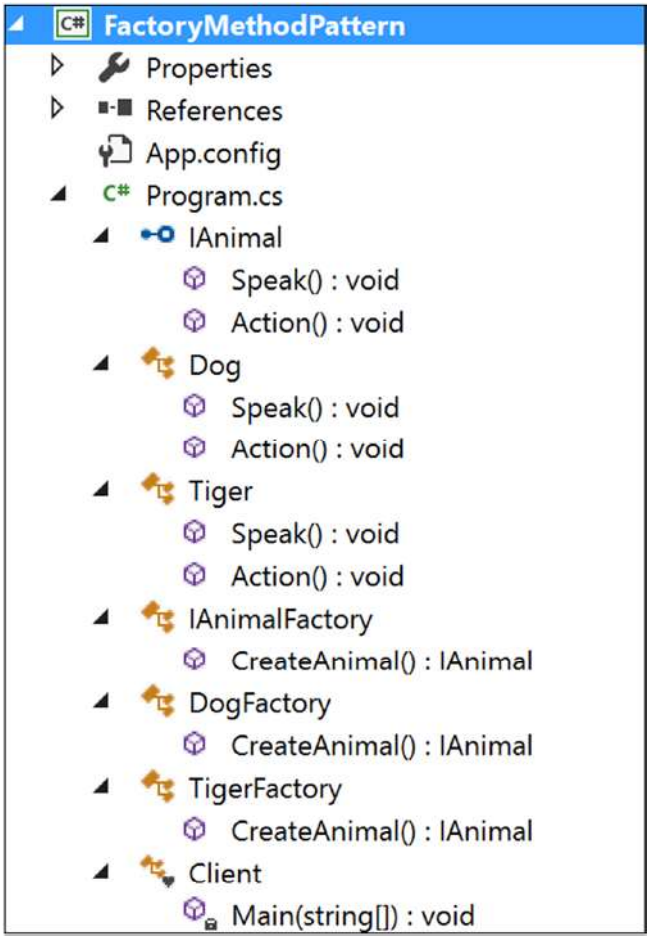


Figure 4-3. *Solution Explorer View*

Implementation

Here is the implementation:

```
using System;

namespace FactoryMethodPattern
{
    public interface IAnimal
    {
        void Speak();
        void Action();
    }
```


www.EBooksWorld.ir

```
    public class Dog : IAnimal
    {
        public void Speak()
        {
            Console.WriteLine("Dog says: Bow-Wow.");
        }
        public void Action()
        {
            Console.WriteLine("Dogs prefer barking...\n");
        }
    }
    public class Tiger : IAnimal
    {
        public void Speak()
        {
            Console.WriteLine("Tiger says: Halum.");
        }
        public void Action()
        {
            Console.WriteLine("Tigers prefer hunting...\n");
        }
    }
    public abstract class IAnimalFactory
    {
//Remember the GoF definition which says "....Factory method lets a class
//defer instantiation to subclasses." Following method will create a Tiger
//or Dog But at this point it does not know whether it will get a Dog or a
//Tiger. It will be decided by the subclasses i.e.DogFactory or TigerFactory.
//So, the following method is acting like a factory (of creation).
public abstract IAnimal CreateAnimal();
    }
    public class DogFactory : IAnimalFactory
    {
        public override IAnimal CreateAnimal()
        {
```


www.EBooksWorld.ir

```
            //Creating a Dog
            return new Dog();
        }
    }
    public class TigerFactory : IAnimalFactory
    {
        public override IAnimal CreateAnimal()
        {
            //Creating a Tiger
            return new Tiger();
        }
    }
    class Client
    {
        static void Main(string[] args)
        {
            Console.WriteLine("***Factory Pattern Demo***\n");
            // Creating a Tiger Factory
            IAnimalFactory tigerFactory =new TigerFactory();
            // Creating a tiger using the Factory Method
            IAnimal aTiger = tigerFactory.CreateAnimal();
            aTiger.Speak();
            aTiger.Action();

            // Creating a DogFactory
            IAnimalFactory dogFactory = new DogFactory();
            // Creating a dog using the Factory Method
            IAnimal aDog = dogFactory.CreateAnimal();
            aDog.Speak();
            aDog.Action();

            Console.ReadKey();
        }
    }
}
```


www.EBooksWorld.ir

Output

Here is some output:

```
***Factory Pattern Demo***

Tiger says: Halum.
Tigers prefer hunting...

Dog says: Bow-Wow.
Dogs prefer barking...
```

Modified Implementation

In this modified implementation, more flexibilities are added. Notice that, the `IAnimalFactory` class is an abstract class. So, you can take the advantage of using an abstract class. Suppose you want a subclass to follow a rule that can be imposed from its parent (or base) class. I have tested such a scenario in the following design.

Here are the key characteristics of the design:

- Only `IAnimalFactory` is modified, as shown here. In other words, I am introducing a new method called `MakeAnimal()`.

```
//Modifying the IAnimalFactory class.
    public abstract class IAnimalFactory
    {
        public IAnimal MakeAnimal()
        {
            Console.WriteLine("\n IAnimalFactory.MakeAnimal()-You
            cannot ignore parent rules.");
            /*
            At this point, it doesn't know whether it will get a
            Dog or a Tiger. It will be decided by the subclasses
            i.e.DogFactory or TigerFactory. But it knows that it
            will Speak and it will have a preferred way of Action.
            */
```


www.EBooksWorld.ir

```
            IAnimal animal = CreateAnimal();
            animal.Speak();
            animal.Action();
            return animal;
        }
        //So, the following method is acting like a factory
        //(of creation).
        public abstract IAnimal CreateAnimal();
    }
```

- The client code has these changes:

```
class Client
    {
        static void Main(string[] args)
        {
            Console.WriteLine("***Beautification to Factory
            Pattern Demo***\n");
            // Creating a tiger  using the Factory Method
            IAnimalFactory tigerFactory = new TigerFactory();
            IAnimal aTiger = tigerFactory.MakeAnimal();
            //IAnimal aTiger = tigerFactory.CreateAnimal();
            //aTiger.Speak();
            //aTiger.Action();

            // Creating a dog  using the Factory Method
            IAnimalFactory dogFactory = new DogFactory();
            IAnimal aDog = dogFactory.MakeAnimal();
            //IAnimal aDog = dogFactory.CreateAnimal();
            //aDog.Speak();
            //aDog.Action();

            Console.ReadKey();
        }
    }
```


www.EBooksWorld.ir

Modified Output

Here is the modified output:

```
***Beautification to Factory Pattern Demo***

AnimalFactory.MakeAnimal()-You cannot ignore parent rules.
Tiger says: Halum.
Tigers prefer hunting...

AnimalFactory.MakeAnimal()-You cannot ignore parent rules.
Dog says: Bow-Wow.
Dogs prefer barking...
```

Analysis

Notice that in each case you see the following warning: "…You cannot ignore parent rules."

Q&A Session

1. **Why have you separated the `CreateAnimal()` method from client code?**

   Answer:

   This is on purpose. I want the subclasses to create specialized objects. If you look carefully, you will also find that only this "creational part" varies across the products. I discussed this in detail in the "Q&A Session" section of Chapter 24.

2. **What are the advantages of using a factory like this?**

   Answer:

   - You are separating the code that varies from the code that does not vary (in other words, the advantages of using the Simple Factory pattern are still present). This helps you to maintain the code easily.

www.EBooksWorld.ir

- The code is not tightly coupled, so you can add new classes such as `Lion`, `Bear`, and so on, at any time in the system without modifying the existing architecture. In other words, I have followed the "closed for modification but open for extension" principle.

3. **What are the challenges of using a factory like this?**

   Answer:

   If you need to deal with many different types of objects, then the overall performance of the system can be affected.

4. **I am seeing that the Factory Method pattern is supporting two parallel hierarchies. Is this understanding correct?**

   Answer:

   Good catch. Yes, from the class diagram, it is evident that this pattern supports parallel class hierarchies; see Figure 4-4.

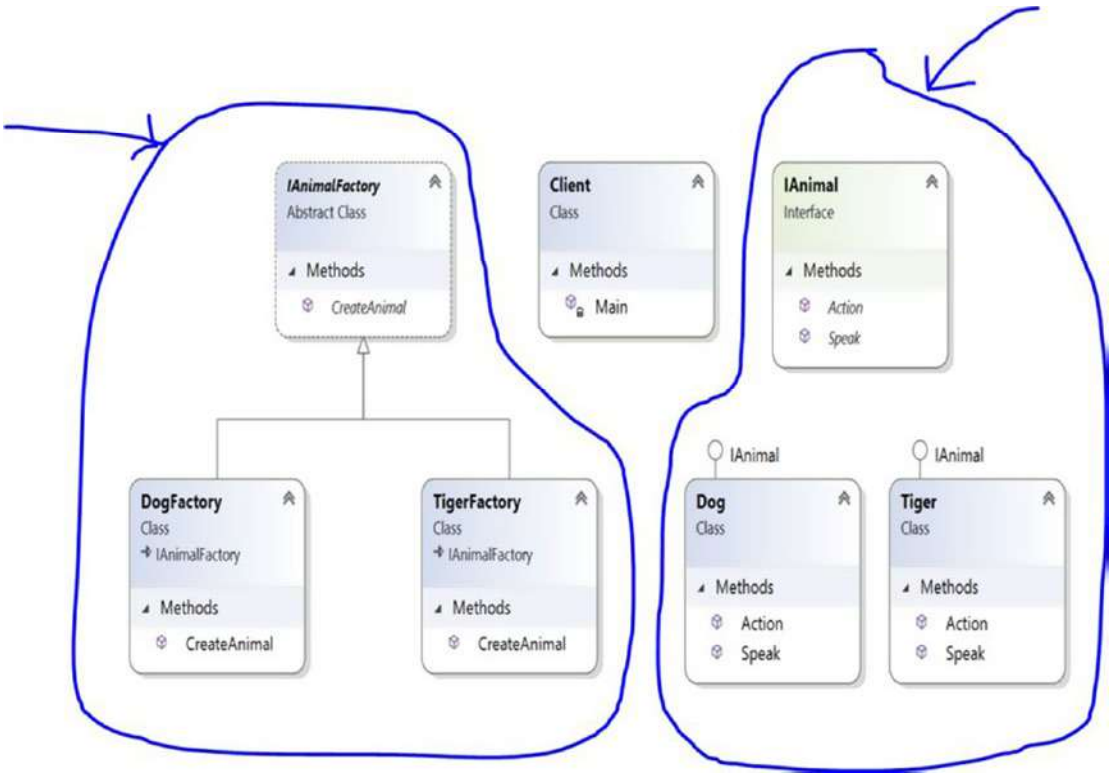


Figure 4-4. *The two class hierarchies in this example*

www.EBooksWorld.ir

In this example, `IAnimalFactory`, `DogFactory`, and `TigerFactory` are placed in one hierarchy, and `IAnimal`, `Dog`, and `Tiger` are placed in another hierarchy. So, you can see that creators and their creations/products are the two hierarchies that are running in parallel.

5. **You should always mark the factory method with an `abstract` keyword so that subclasses can complete them. Is that understanding correct?**

   Answer:

   No. Sometimes you may be interested in a default factory method if the creator has no subclasses. In that case, you cannot mark the factory method with an `abstract` keyword.

   However, to see the real power of the Factory Method pattern, you may need to follow the design that is implemented here.

6. **It still appears to me that the Factory Method pattern is not that much different from the Simple Factory pattern. Is that understanding correct?**

   Answer:

   If you look at the subclasses in the examples in both chapters, you may find some similarities. But you should not forget the key aim of the Factory Method pattern; it is supplying you with the framework through which different subclasses can make different products. In the case of the Simple Factory pattern, you cannot vary the products in a similar manner. You can think of the Simple Factory pattern as a one-time deal, but most important, your creational part will not be closed for modification. Whenever you want to add something new, you need to add an `if-else` block or a `switch` statement in the factory class of your Simple Factory pattern.

www.EBooksWorld.ir

In this context, remember the GoF definition ("The Factory Method pattern lets a class defer instantiation to subclasses."). So, in the Simple Factory pattern demonstration, you could omit the abstract class `IAnimalFactory` and its abstract method `CreateAnimal()` and instead use only one `SimpleFactory` class. In that case, you would not need to override the `CreateAnimal()` method; in addition, it's considered a good practice to code to an interface/abstract class (as in this case). Also, this mechanism provides you with the flexibility to put some common behavior in the abstract class.

www.EBooksWorld.ir